

SQLite Gimnasio Socio Clase Solucion

Proyecto resuelto inspirado en el formato de `employee-skills-sqlite`, adaptado al dominio de gimnasio y centrado en dos bloques de servicio:

- `SocioService`
- `ClaseService`

Objetivo

Practicar servicios y repositorios sobre SQLite3 con:

- operaciones CRUD en servicios
- consultas con `WHERE`
- consultas con `JOIN/WHERE` entre tablas sencillos

Base de datos

Fichero principal:

```
src/main/resources/data/sqlite/gimnasio.db
```

Copia de respaldo:

```
src/main/resources/data/sqlite/gimnasio_backup.db
```

Esquema y datos semilla:

```
src/main/resources/data/sqlite/gimnasio_schema.sql
```

Comandos utiles sqlite3

Abrir la base de datos:

```
sqlite3 src/main/resources/data/sqlite/gimnasio.db
```

Mostrar tablas:

```
.tables
```

Mostrar esquema:

```
.schema
```

Mostrar columnas:

```
PRAGMA table_info(socio);  
PRAGMA table_info(clase);  
PRAGMA table_info(reserva);
```

Servicios implementados

SocioService

- create
- findById
- findAll
- update
- deleteById
- findActivos
- findByPlan
- findSociosConReservas

ClaseService

- create
- findById
- findAll
- update
- deleteById
- findDisponibles
- findByTipo
- findByMonitor
- findClasesConMonitor
- findReservasConSocio

Patron de tests

Todos los tests estan hechos sobre servicios y siguen este patron:

- findByIdOkTest
- findByIdNullTest
- findByIdEmptyTest
- findByIdFailTest

Cada funcion del servicio tiene al menos 4 tests.

Los tests usan:

- `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`
- `@Order(...)`

Ejecutar tests

```
mvn test
```

Calcular nota

```
mvn clean verify -Pcalificar
```

La nota se calcula solo a partir de los tests de `service` y la documentacion de las interfaces de servicio.

Cobertura

Tras ejecutar tests, el informe JaCoCo queda en:

```
target/site/jacoco/index.html
```

Salida esperada de la base de datos

Esta sección documenta los datos semilla esperados en `gimnasio.db` y las salidas que deben devolver las consultas principales usadas por los servicios.

Tablas esperadas

```
clase  
monitor  
reserva  
socio
```

Datos esperados: `socio`

id	dni	nombre	email	telefono	plan	activo
1	44444444D	Ana Ruiz	ana@gym.com	600111111	premium	1
2	55555555E	Luis Vega	luis@gym.com	600222222	basic	1
3	66666666F	Carla Sol	carla@gym.com	600333333	vip	0

Datos esperados: `monitor`

id	dni	nombre	especialidad	activo
1	11111111A	Laura Coach	Yoga	1
2	22222222B	Pablo Fit	Spinning	1
3	33333333C	Marta Power	Crossfit	1

Datos esperados: **clase**

id	nombre	tipo	horario	cupo_maximo	plazas_disponibles	activa	id_monitor
1	Yoga Manana	yoga	2026-05-01 09:00:00	20	5	1	1
2	Spinning Tarde	spinning	2026-05-01 18:00:00	15	0	1	2
3	Crossfit Pro	crossfit	2026-05-02 19:00:00	12	2	1	3
4	Pilates Suave	pilates	2026-05-03 10:00:00	18	8	0	1

Datos esperados: **reserva**

id	fecha	estado	id_socio	id_clase
1	2026-04-20 10:00:00	reservada	1	1
2	2026-04-20 10:15:00	asistida	2	2
3	2026-04-21 11:00:00	cancelada	1	3

Salidas esperadas por método de servicio

SocioService.findAll()

Debe devolver todos los socios:

id	dni	nombre	email	telefono	plan	activo
1	44444444D	Ana Ruiz	ana@gym.com	600111111	premium	1
2	55555555E	Luis Vega	luis@gym.com	600222222	basic	1
3	66666666F	Carla Sol	carla@gym.com	600333333	vip	0

SocioService.findById(1)

Debe devolver:

id	dni	nombre	email	telefono	plan	activo
1	44444444D	Ana Ruiz	ana@gym.com	600111111	premium	1

`SocioService.findActivos()`

Debe devolver solo socios activos (`activo = 1`):

id	dni	nombre	email	telefono	plan	activo
1	44444444D	Ana Ruiz	ana@gym.com	600111111	premium	1
2	55555555E	Luis Vega	luis@gym.com	600222222	basic	1

`SocioService.findByPlan("premium")`

Debe devolver los socios del plan `premium`:

id	dni	nombre	email	telefono	plan	activo
1	44444444D	Ana Ruiz	ana@gym.com	600111111	premium	1

`SocioService.findSociosConReservas()`

Debe devolver socios combinados con sus reservas mediante `JOIN`:

id_socio	socio	plan	id_reserva	fecha	estado	id_clase
1	Ana Ruiz	premium	1	2026-04-20 10:00:00	reservada	1
1	Ana Ruiz	premium	3	2026-04-21 11:00:00	cancelada	3
2	Luis Vega	basic	2	2026-04-20 10:15:00	asistida	2

`ClaseService.findAll()`

Debe devolver todas las clases:

id	nombre	tipo	horario	cupos_maximo	plazas_disponibles	activa	id_monitor
1	Yoga Manana	yoga	2026-05-01 09:00:00	20	5	1	1
2	Spinning Tarde	spinning	2026-05-01 18:00:00	15	0	1	2

id	nombre	tipo	horario	cupo_maximo	plazas_disponibles	activa	id_monitor
3	Crossfit Pro	crossfit	2026-05-02 19:00:00	12	2	1	3
4	Pilates Suave	pilates	2026-05-03 10:00:00	18	8	0	1

ClaseService.findById(1)

Debe devolver:

id	nombre	tipo	horario	cupo_maximo	plazas_disponibles	activa	id_monitor
1	Yoga Manana	yoga	2026-05-01 09:00:00	20	5	1	1

ClaseService.findDisponibles()

Debe devolver clases activas con plazas disponibles (**activa = 1** y **plazas_disponibles > 0**):

id	nombre	tipo	horario	cupo_maximo	plazas_disponibles	activa	id_monitor
1	Yoga Manana	yoga	2026-05-01 09:00:00	20	5	1	1
3	Crossfit Pro	crossfit	2026-05-02 19:00:00	12	2	1	3

ClaseService.findByTipo("yoga")

Debe devolver las clases de tipo **yoga**:

id	nombre	tipo	horario	cupo_maximo	plazas_disponibles	activa	id_monitor
1	Yoga Manana	yoga	2026-05-01 09:00:00	20	5	1	1

ClaseService.findByMonitor(1)

Debe devolver las clases asignadas al monitor con id **1**:

id	nombre	tipo	horario	cupo_maximo	plazas_disponibles	activa	id_monitor
1	Yoga Manana	yoga	2026-05-01 09:00:00	20	5	1	1
4	Pilates Suave	pilates	2026-05-03 10:00:00	18	8	0	1

ClaseService.findClasesConMonitor()

Debe devolver clases combinadas con su monitor mediante **JOIN**:

id_clase	clase	tipo	horario	id_monitor	monitor	especialidad
1	Yoga Manana	yoga	2026-05-01 09:00:00	1	Laura Coach	Yoga
2	Spinning Tarde	spinning	2026-05-01 18:00:00	2	Pablo Fit	Spinning
3	Crossfit Pro	crossfit	2026-05-02 19:00:00	3	Marta Power	Crossfit
4	Pilates Suave	pilates	2026-05-03 10:00:00	1	Laura Coach	Yoga

ClaseService.findReservasConSocio()

Debe devolver reservas de clases combinadas con socios mediante **JOIN**:

id_reserva	fecha	estado	clase	id_socio	socio	plan
1	2026-04-20 10:00:00	reservada	Yoga Manana	1	Ana Ruiz	premium
2	2026-04-20 10:15:00	asistida	Spinning Tarde	2	Luis Vega	basic
3	2026-04-21 11:00:00	cancelada	Crossfit Pro	1	Ana Ruiz	premium

ada funcion del servicio tiene al menos 4 tests.

Los tests usan:

- `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`
- `@Order(...)`

Ejecutar tests

```
mvn test
```

Calcular nota

```
mvn clean verify -Pcalificar
```

La nota se calcula solo a partir de los tests de **service** y la documentacion de las interfaces de servicio.

Cobertura

Tras ejecutar tests, el informe JaCoCo queda en:

```
target/site/jacoco/index.html
```

RESULTADOS:

SOCIO

```
- Tests: 32/32 pasados, 0 fallados (100%)
- Nota tests: 4.00/4.00
- Documentacion encontrada: 8/8 metodos (100%)
- Nota documentacion: 1.00/1.00
- Interfaz completa para este bloque
- Subtotal: 5.00/5.00
```

CLASE

```
- Tests: 40/40 pasados, 0 fallados (100%)
- Nota tests: 4.00/4.00
- Documentacion encontrada: 10/10 metodos (100%)
- Nota documentacion: 1.00/1.00
- Interfaz completa para este bloque
- Subtotal: 5.00/5.00
```

=== NOTA FINAL ===

Nota final: 10.00/10